



Como evoluir o Radar

Princípios, receitas de extensão, escala e próximos passos

Radar do Cotador · proposta para o Cotador de Planos de Saúde

Código e documentação: github.com/GustavoHenriqueRS/radar-cotador

Protótipo no navegador: gustavohenriquers.github.io/radar-cotador

Resumo em vídeo (41 s): gustavohenriquers.github.io/radar-cotador/video/radar-cotador-motion.mp4

Demonstração completa (1 min 36 s): gustavohenriquers.github.io/radar-cotador/video/radar-cotador-demo.mp4

Sumário

1. Princípios que não mudam

2. Receitas

2.1 Fonte nova de um tipo de captura que já existe

2.2 Tipo de captura novo (API de parceiro, SFTP, Google Drive, portal com credencial autorizada)

2.3 Layout de PDF que o leitor não entende

2.4 Regra nova de conferência

2.5 Dado aberto novo da ANS

2.6 Evento novo no radar

3. Como escala

4. O que vem depois, em ordem

5. Operação

Publicação

Este guia é para quem vai manter e aumentar o sistema: o que não pode mudar, como fazer cada tipo de extensão, como escala e o que vem depois. A proposta (docs/proposta.md) explica o porquê; aqui fica o como.

1. Princípios que não mudam

1. **O mesmo material dá sempre o mesmo resultado.** A leitura geométrica, o OCR, o cálculo pelo padrão de faixas, as regras da ANS, o casamento de versões e o histórico são determinísticos (há teste que lê o mesmo PDF duas vezes e compara byte a byte, tests/test_determinismo.py).
2. **O LLM é segunda opinião, nunca publica sozinho.** Cada leitura por LLM fica gravada pelo hash do PDF, com o modelo, o custo e a data. Refazer a conferência com a gravação dá o mesmo resultado, então uma publicação pode ser auditada meses depois.
3. **Nada é sobrescrito.**
 - O PDF original é guardado e vira a prova da origem.
 - Cada publicação é uma versão nova.
 - Material mais antigo que o vigente entra no histórico e nunca volta a ser o preço da cotação, em qualquer ordem que os documentos cheguem.
4. **Produto se identifica pelo registro ANS, nunca pelo nome.** O nome se repete; o registro, não.
5. **Fonte é configuração.** Quem publica e por onde o material chega ficam em fontes/*.json , revisados como código.
6. **A fonte manda.**
 - O robots.txt é obedecido, inclusive o do site original quando a cópia vem do arquivo da web.
 - Nada de login, captcha ou WAF contornado.
 - Onde não há canal aberto, o caminho é e-mail ou parceria.
7. **Toda dúvida vira célula apontada, não palpite.**
 - Divergência entre leituras, regra da ANS violada ou cálculo que não fecha vão para uma pessoa, com o PDF ao lado.
 - O valor calculado aparece como sugestão, nunca como correção automática.

2. Receitas

2.1 Fonte nova de um tipo de captura que já existe

Um arquivo em fontes/. Exemplo de uma administradora que publica tabelas numa página:

```
{
  "fontes": [{
    "nome": "Administradora X",
    "tipo": "pdf_administradora",
    "url": "https://www.administradorax.com.br/tabelas",
    "confiabilidade": 4,
    "observacoes": "robots.txt permite /tabelas; data da versão no nome do arquivo",
    "capturas": [{
      "tipo": "pagina_publica",
      "nome": "Página de tabelas",
      "ativa": true,
      "config": {
        "url": "https://www.administradorax.com.br/tabelas",
        "incluir": "/tabelas/.*\\.pdf$"
      }
    }]
  }]
}
```

Depois:

```
PYTHONPATH=. .venv/bin/python -m coletor.fontes
```

```
cd backend && ../../.venv/bin/python manage.py carregar_fontes && ../../.venv/bin/python manage.py coletar --fonte "Administradora X"
```

O validador recusa:

- tipo de fonte ou de captura desconhecido;
- campo obrigatório faltando;
- expressão regular inválida.

Nada é gravado se algum arquivo tiver problema. Os tipos disponíveis e a configuração de cada um estão na docstring da classe, em `coletor/`.

Tipo	Quando usar	Configuração mínima
<code>pagina_publica</code>	página que lista os PDFs	<code>url</code> ; <code>atributos</code> se o link não estiver em <code>href</code>
<code>url_direta</code>	endereço fixo que a fonte substitui no lugar	<code>urls</code>
<code>api_json</code>	página feita em JavaScript que monta a lista a partir de uma API aberta	<code>url</code> , <code>campo_url</code> ; <code>lista</code> e <code>campo_titulo</code> conforme a resposta
<code>wayback</code>	versões que a fonte já tirou do ar	<code>enderecos</code> (URL ou prefixo terminado em <code>/</code>); <code>somente_series_conhecidas</code> para trazer só o histórico do que já se acompanha
<code>caixa_email</code>	operadora que manda o material por e-mail ou bloqueia robôs	<code>servidor</code> , <code>usuario</code> , <code>senha_env</code> , <code>remetentes</code>

2.2 Tipo de captura novo (API de parceiro, SFTP, Google Drive, portal com credencial autorizada)

O tipo `api_json` é o exemplo real: o mapeamento da Hapvida achou 53 tabelas da CORPe numa API aberta que nenhum tipo lia, e a classe nova tem 40 linhas (`coletor/api_json.py`).

Uma classe em `coletor/` que herda de `Coletor` e só sabe descobrir candidatos:

```
class ApiParceiro(Coletor):
    tipo = "api_parceiro"
    rotulo = "API de parceiro"
    obrigatorios = ("url", "token_env")
    imutavel = False          # o mesmo endereço pode mudar de conteúdo
    lista_completa = True     # a descoberta lista tudo o que a fonte oferece agora

    def descobrir(self, config, http, estado):
        ...
        return [Candidato(url=..., titulo=..., data-versao=...)]
```

O motor faz o resto para qualquer tipo:

- robots.txt, intervalo entre pedidos e download condicional;
- deduplicação por hash e ligação com a versão anterior (a série);
- validação do PDF, leitura, avisos no radar e histórico de cada coleta.

Se o tipo já traz o arquivo junto (como o e-mail), basta preencher `Candidato.conteudo`. Registre a classe em `coletor/__init__.py` (TIPOS) e escreva o teste com um servidor falso (veja `ImapFalso` em `tests/test_coletor.py`). As escolhas de `Captura.tipo` vêm de `TIPOS`: rode `makemigrations` para registrá-las.

`estado` atravessa as coletas (o e-mail guarda o último UID lido) e só é gravado se a coleta terminar bem. Credencial nunca vai para o JSON: a configuração diz o nome da variável de ambiente.

2.3 Layout de PDF que o leitor não entende

1. Rode o leitor e olhe a saída:

```
.venv/bin/python -m leitor.cli caminho/do.pdf
```

Ele grava `saidas/<nome>.json` com cada coluna, célula, posição e achado.

2. Classifique o problema:

- **sem preço lido**: a âncora das faixas não foi achada; olhe os rótulos em `leitor/faixas.py`;
- **coluna sem registro ANS**: o cabeçalho está longe ou quebrado; olhe `_associar_cabecalho` em `leitor/geometrico.py`;
- **condição de venda errada**: o vocabulário de condições está em `leitor/condicoes.py`;
- **PDF sem texto**: vai para o OCR sozinho; tabela em imagem dentro de PDF com texto passa pela leitura por LLM (ver a seção 4).

3. Corrija no leitor genérico, nunca com um caso especial para aquele arquivo.

4. Acrescente o PDF em `amostras/` com a origem no `manifest.json` e um teste que fixa o caso.

5. Rode o acervo inteiro (`pytest`) para ver que nenhum outro layout piorou.

Enquanto a correção não sai, o material continua entrando: a leitura por LLM cobre o layout, e a divergência com a geométrica vai para revisão.

2.4 Regra nova de conferência

As regras ficam em `leitor/regras.py`. Cada uma recebe a coluna lida e devolve achados (`erro`, `alerta` ou `info`) com a base legal no nome. O caminho é:

1. Escrever a regra contra um caso real do acervo.
2. Rodar o acervo inteiro e contar os alarmes. Regra que acusa preço certo não entra.
3. Fixar o caso em teste.

Os três cuidados que os dados reais ensinaram:

- tolerância de centavos (as operadoras usam o limite exato);
- reajuste uniforme não é erro;
- preço composto (com odonto) não segue proporção.

2.5 Dado aberto novo da ANS

O padrão está em `leitor/rede.py`:

- baixar só o trecho necessário (HTTP Range no zip, por UF);
- descompactar em fluxo, com o filtro barato sobre os bytes antes do parser;

- guardar só o que interessa num SQLite local, com a data da fonte;
- ligar ao radar por uma função `sincronizar_*` que gera eventos e não repete aviso.

Conjuntos que valem a pena, em ordem, todos no portal de dados abertos da ANS:

Conjunto	Para quê
<code>area_comercializacao_planos_ntrp</code>	municípios onde cada plano pode ser vendido: confere a região da tabela e filtra a cotação pela cidade do cliente
<code>operadoras_e_prestadores_nao_hospitalares</code>	laboratórios e clínicas da rede, no mesmo modelo da rede hospitalar
<code>percentuais_de_reajuste_de_agrupamento</code>	reajuste do pool de contratos PME de cada operadora: prevê a próxima tabela
<code>servicos_opcionais_planos_saude</code>	serviços opcionais registrados por plano
<code>historico_planos_saude</code>	mudanças de situação do plano ao longo do tempo

2.6 Evento novo no radar

1. Um valor em `Evento.Tipo` com a migração.
2. O ícone em `frontend/src/components/Eventos.tsx`.
3. A função que gera o evento, com chave de deduplicação no `detalhe` para não repetir aviso, como `sincronizar_rede` faz com o protocolo da ANS.

Severidade:

- `erro` : não pode ir para a cotação;
- `alerta` : alguém precisa agir;
- `info` : registro do que mudou.

3. Como escala

Hoje (protótipo)	Em produção	Quando trocar
Thread em segundo plano por documento	Fila (Celery ou RQ com Redis), um documento por tarefa	Desde o início em produção: o documento é a unidade de trabalho e os leitores não guardam estado
Agendador do docker compose (coletar uma vez por dia)	Celery beat por captura, com horário e frequência por fonte	Ao passar de uma dezena de fontes
Índices da ANS em SQLite local	Tabelas no PostgreSQL, atualizadas por job	Quando houver mais de um servidor de aplicação
PDFs em disco local	Armazenamento de objetos (S3 ou equivalente), nunca apagados	Desde o início em produção
LLM por chamada direta	A mesma chamada, com fila e limite de gasto por dia	Ao passar de alguns mil documentos por mês

Custo medido:

Item	Custo	Quando
Segunda leitura (GPT-6 Luna)	cerca de US\$ 0,001 por página	toda página nova; documento repetido não é relido (hash)
Modelo forte (GPT-6.1 Sol)	18 vezes o Luna	só no PDF escaneado ou na página com divergência
Rede hospitalar dos 392 planos acompanhados	5,6 minutos	por mês
Índice da ANS	21 segundos	por dia

4. O que vem depois, em ordem

1. API do Cotador e saúde da base atual.

- O Cotador consome as tabelas publicadas, com o selo de origem.
- O que ele já tem cadastrado é cruzado com a ANS: operadoras canceladas, planos suspensos, registros que não existem.

2. Parcerias e canais fechados.

- Caixa de e-mail dedicada para cada operadora que manda material.
- Envio pelo corretor, com crédito para quem colabora.
- Feeds de administradoras (o tipo de captura é o mesmo contrato).

3. Série reconhecida pelo conteúdo.

- Hoje a versão nova se liga à anterior pelo endereço sem a data.
- Quando a fonte muda a pasta ou o nome do arquivo (a Allcare mudou os dois), o conjunto de registros ANS do documento identifica a série. É o “classificador” que encaminha um PDF sem endereço estável, como o anexo de e-mail.

4. Preço único sem faixa etária.

- Alguns materiais PME trazem um valor só por vida.
- O modelo de dados publica hoje só tabela por faixa; esses materiais vão para revisão.

5. OCR por página.

- A decisão de OCR é por documento.
- Tabela em imagem dentro de um PDF com texto passa só pelo LLM.

6. Reembolso com valores.

- A ANS registra se o plano tem livre escolha (reembolso) e a cotação mostra isso.
- Os valores e múltiplos de reembolso vêm do material. A leitura por LLM já extrai essas condições; o passo seguinte é um campo estruturado com regra de conferência.

7. Rede não hospitalar e área de comercialização, no mesmo padrão da rede hospitalar (seção 2.5).

5. Operação

Rotina	Comando	Frequência
Coleta das capturas ativas	<code>manage.py coletar</code>	diária (perfil <code>coleta</code> do docker compose)
Conferência contra a ANS (operadoras, planos, vigências, rede)	<code>manage.py sincronizar_ans</code>	diária

Rotina	Comando	Frequência
Rede hospitalar e mudanças de rede	<code>manage.py</code> <code>baixar_rede_ans</code>	mensal (a ANS atualiza a rede uma vez por mês)
Fontes e capturas	<code>manage.py</code> <code>carregar_fontes</code>	a cada mudança em fontes/

O que acompanhar:

- coletas com erro ou bloqueadas;
- página que listava tabelas e passou a listar nenhuma (o radar avisa);
- células em revisão por fonte;
- custo do LLM por dia;
- idade média do material por trás das cotações.

Publicação

O protótipo completo roda numa EC2 da AWS com Docker Compose e sai por um túnel da Cloudflare (`deploy/subir-com-tunnel.sh` , explicado no `README.md`). Com mais operadoras e mais corretores usando, a publicação cresce assim:

- domínio próprio, com um túnel nomeado da Cloudflare ou um balanceador da AWS na frente;
- PostgreSQL gerenciado, com backup automático, no lugar do contêiner;
- os PDFs originais num armazenamento de objetos (S3 ou equivalente), com versionamento, porque o PDF é a prova da origem;
- a leitura numa fila (Celery ou RQ), separada do servidor web, para um lote grande de PDFs não pesar na tela de revisão;
- o deploy automático a cada mudança na branch principal, depois do `./verificar.sh` .